Database Group Project — Deliverables and Demonstration Requirements

Initial Deliverables

- **Functional Requirements Document (FRD)**
    - Define the business problem, project purpose, scope, stakeholders, user roles, functional requirements, business rules, workflow/use cases, reporting needs, assumptions, and acceptance criteria.
    - Requirements must be traceable to the data model, schema objects, security/entitlement rules, test cases, and final demo evidence.
    - Use clear, testable language: identify the actor, action, business condition, expected result, and relevant data.
- **Technical Design Document (TDD)**
    - Explain how the functional requirements will be implemented in the database.
    - Include the items in the **Technical Design Expectations** section below.
- **Data Model / ER Diagram (draw.io)**
    - Show all entities, primary keys, foreign keys, attributes, relationship cardinalities, and associative entities.
    - Identify lookup/reference tables and distinguish required from optional relationships where relevant.
    - Keep the ERD consistent with the FRD, TDD, and implemented schema.
- **Schema Build File**
    - Provide a repeatable SQL script that creates the database/schema and all required implementation objects.
    - Include tables, keys, constraints, indexes, views, stored procedures/functions, triggers, roles/privileges, and other required database objects.
    - The script must be safe to run in the documented sequence on a fresh database environment.
- **Repository Build and Initialization Script**

- Store project materials in the team repository with clear setup instructions.
- Provide a documented script or ordered script sequence that extracts/clones the repository materials, builds the schema, and loads initial/sample data. **Suggest that you use a version label to extract build objects out of the repo**
- A new user should be able to follow the README and create a working project database without manually inventing missing steps.

Technical Design Expectations

- **Traceability**
  - Map each significant FRD requirement to the relevant entity/table, constraint, procedure, trigger, view/query, entitlement rule, and test case.
  - Include a traceability matrix such as:

    `FR requirement → schema objects → enforcement method → test case → demo evidence`

- **Testing and Test Evidence**
  - Define positive, negative, boundary, and exception test cases for significant business rules.
  - Show tests for primary-key, foreign-key, unique, `NOT NULL`, `CHECK`, and status/domain validation rules where applicable.
  - Demonstrate that unauthorized actions, invalid workflow transitions, duplicate records, or other invalid transactions are rejected.
  - For important multi-step operations, show that a failure rolls back incomplete work and does not leave partial or orphaned data.
  - Include expected result, actual result, SQL/procedure executed, and query evidence for each major test.
- **Roles, Entitlements, and Data Access**
  - Define business roles and the entitlements associated with each role.
  - Clearly identify which roles may view, create, update, delete/cancel, approve, administer, and audit each protected data area.

    - Address row-level access when relevant; for example, an owner may manage only owned records and a participant may act only on assignments specifically assigned to that participant.
    - Identify how authorization is enforced: database roles/grants, views, stored procedures, application logic, or a combination.
    - Protect completed approvals, signatures, finalized records, and audit history from unauthorized modification.
- **Performance and Index Design**
    - Identify the common operational queries, reports, filters, joins, sort orders, and high-volume access paths that drive index choices.
    - Document each important index, its indexed columns, the query pattern it supports, and the expected benefit.
    - Avoid adding indexes without a documented query or integrity reason; explain meaningful trade-offs for write-heavy tables.
    - Use realistic sample data sufficient to demonstrate query behavior, joins, filtering, and index usage.
- **Resiliency, Recovery, and Data Integrity**
    - Identify transactions that must be atomic: either every related insert/update/audit event succeeds, or all related work is rolled back.
    - Explain how the design prevents duplicate submissions, conflicting updates, invalid state transitions, and partial transactions.
    - Preserve history for business records that require traceability; prefer controlled status changes or soft deletion when hard deletion would damage referential integrity or audit history.
    - Define protection for immutable/finalized records and append-only audit records.
    - Describe how the system handles cancellation, expiration, failure, correction, recovery, and retry scenarios.
- **Audit and Traceability Design**

- Identify the business events that must be audited, such as creation, updates, status transitions, assignments, approvals, signatures, cancellations, administrative changes, and security-sensitive failures.
- Define audit content: actor/user, timestamp, event type, affected entity and record identifier, transaction/context identifier, and meaningful prior/new values where appropriate.
- Explain how audit records are protected from update or deletion.
- Show how the audit trail can reconstruct the history of a selected business record or workflow.

Required Final Demonstration

- **Fresh Build and Data Initialization**
  - Start with a clean or newly created database environment.
  - Demonstrate the documented repository/setup process.
  - Run the schema build script(s) and show successful creation of the database objects.
  - Run the data-initialization script(s) and show that required reference data and realistic sample data were loaded.
  - Run verification queries showing expected tables, constraints, indexes, users/roles if applicable, and row counts.
- **Core Business Workflows**
  - Demonstrate the main end-to-end workflow(s) defined in the FRD.
  - Show successful creation, retrieval, update, and controlled status progression for core business records.
  - Demonstrate at least one valid transaction and one invalid/unauthorized/invalid-state transaction that is correctly rejected.
  - Show the resulting data before and after the transaction, including related rows and status changes.
- **Common Queries and Performance Evidence**
  - Run representative operational and reporting queries identified in the FRD/TDD.
  - Use `EXPLAIN` or the database platform's equivalent execution-plan feature for important queries.

    - Explain the access path: tables joined, predicates used, index selected, estimated/actual rows where available, and whether the plan performs a full scan, index lookup, range scan, sort, or temporary operation.
    - Show how the selected indexes support the important queries; if an index is not used, explain why and whether a design change is appropriate.
    - Demonstrate at least one query involving joins, filtering, ordering, aggregation, or exception reporting.
- **Entitlement Demonstration**
    - Demonstrate that an authorized role can perform an allowed protected action.
    - Demonstrate that an unauthorized role or unassigned user cannot perform the same action.
    - Show any row-level restrictions, protected views, controlled procedures, role grants, or other enforcement mechanisms used by the project.
- **Audit and Traceability Demonstration**
    - Perform one or more audited business actions during the demo.
    - Query the audit/history records and show the actor, event type, timestamp, affected record, and relevant context or changed values.
    - Demonstrate that completed/finalized records and audit records cannot be silently altered or deleted.
    - Show a history query/report that traces a selected record through its lifecycle, related transactions, and significant status changes.
- **Resiliency Demonstration**
    - Demonstrate one controlled failure or exception scenario for a multi-step transaction.
    - Show that invalid or incomplete work is rolled back or otherwise leaves the database in a valid, consistent state.
    - Demonstrate the project's treatment of cancellation, expiration, recovery, retry, or correction where applicable.

Submission Quality Checklist

- All deliverables use consistent entity names, keys, statuses, terminology, and requirement identifiers.

- The implemented schema matches the ERD and the functional/technical documentation.
- All scripts run from a fresh environment in the documented order.
- The repository README identifies prerequisites, platform/version assumptions, execution order, credentials or sample accounts if needed, and verification steps.
- Demo evidence is reproducible: another person can run the build, initialize the data, execute the tests, and reproduce the principal queries and audit trail.